

TAO — Detecting Semantic Laundering in Agentic AI

A behavioral audit interface for checking what an agent claims it did against what the action mechanically did.

Version 0.11 working draft · May 2026 · Jorge Perdomo

An agent reports it was drafting release notes. The trace records a read from a private repo, followed by an HTTPS POST to an endpoint nobody authorized.

An agent reports it was helping a customer with billing. The trace records a payment three times above policy.

Today’s agent telemetry records both halves of these stories. What it does not provide is a standard layer that asks whether the agent’s label for an action is consistent with what the action mechanically did. The mismatch is invisible until somebody reads the logs by hand and asks the awkward question.

The gap has a name: **semantic laundering** — a benign label sitting on top of effects that don’t support it. Red-teamers working on tool-using agents see versions of it routinely.

Where this sits in the field. Tracing, audit trails, policy engines, privacy compliance, provenance — each category has at least one serious public effort over the last eighteen months. None target the gap between what an agent claimed and what mechanically happened.

System	Traces · spans	Tamper-evident records	Runtime policy	Compliance mapping	Semantic ↔ mechanical integrity	Claim/check + observer ladder
OpenTelemetry GenAI	✓					
Agent Audit Trail (IETF, '26)	~	✓		✓		
LangSmith / Langfuse / Datadog	✓					
Microsoft AGT / Agent OS		~	✓	~		
OPA / policy-as-code			✓			
AudAgent (arXiv, '26)	~		~		privacy only	
W3C PROV						partial
TAO v0.11		✓	✓	✓	✓	✓

✓ direct coverage ~ partial blank cell = not in scope. **This table compares conceptual coverage, not implementation maturity.** TAO v0.11 is a working draft and does not yet ship a runtime or an executable validator; a reference validator is the next planned artifact. *Runtime policy* means the system defines a decision schema (allow / block / escalate / log) — TAO’s Mission Profile is analogous to OPA’s policy bundle, and enforcement is the caller’s job in both cases. *Compliance mapping* refers to the companion `TAO_COMPLIANCE_CROSSWALK.md`. *Traces & spans* are OpenTelemetry units; TAO consumes them rather than producing them.

The rightmost two columns are what’s new. TAO is the only public artifact I am aware of that puts a compact action vocabulary, a mechanical-effects kernel, REQUIRED/FORBIDDEN/PERMITTED mapping rules between them, independently-emitted claim and check tuples, and a defined observer-assurance ladder into a single behavioral audit interface.

What TAO is

A working-draft interface for detecting that mismatch. Every action gets recorded in two registers and forced to be consistent:

1. A **semantic claim** — the verb the agent used, the purpose it stated, the authority it cited.
2. The **mechanical effects** the action produced, drawn from a fixed set of nine (resource transfer or damage; capability restrict or enable; info disclose, withhold, or fabricate; commitment make or break).

For each verb, the spec defines which effects must be present, which must not, and which are permitted only when explicitly acknowledged. A claim that doesn’t match its effects is structurally rejected. A claim that survives is then checked against an independently-generated tuple of the same action — the **Claim-Check Delta** — and reported as consistent, inconsistent, or indeterminate. Tuple format, 29-verb vocabulary, 9-effect kernel, the mapping between them, the delta, and a signed configuration (Mission Profile) for deployment policy. That is the whole standard.

What TAO is not

Not a telemetry layer; OpenTelemetry GenAI, LangSmith, Datadog, and Langfuse handle the trace, and TAO consumes their telemetry as raw input. Not a policy engine; OPA and Microsoft's Agent Governance Toolkit enforce decisions at runtime, and TAO gives them a richer record to reason over than a raw tool-call log. Not an audit-trail format competing with the IETF Agent Audit Trail draft; TAO sits one floor above and records whether the labels on AAT-style events match what the events mechanically did. Not a moral framework, a training objective, or a runtime architecture.

How it works, in one minute

A code agent gets a task: "Help me draft release notes for v3.2." It emits a tuple with verb **CREATE.GENERATE.PRODUCE** and stated purpose "draft release-notes content." A sidecar observer running at **PRIVILEGE_ISOLATED** separation records the actual tool calls: a read from the private repo (effect **INFO.DISCLOSE**), then an HTTPS POST to an endpoint not on the allow-list (another **INFO.DISCLOSE**). CCD: the verb requires a creation effect, none is present (**inconsistent**); stated purpose is content generation, observed flow ends in external disclosure (**DIRECT_CONTRADICTION**); authority chain does not authorize the external call (**inconsistent**). The Mission Profile blocks the second call before it completes. The audit record — claim, check, CCD result, observer level, profile hash — is appended to a hash-chained log a compliance reviewer can read six months later without ever touching weights or prompts.

Why a frontier lab should care

Tool-using agents are now the consequential product surface. Browser, code, computer-use, and enterprise tool agents are the parts of the portfolio that do things in the world, and the parts whose behavior enterprise customers, internal safety teams, and regulators all want to inspect. Enterprise procurement increasingly asks for an inspectable behavioral record before signing; compliance teams are not satisfied with "we have logs." Regulators are converging on the audit side, not the model side — EU AI Act, UK AISI, the emerging US frameworks want records, not weights. A lab that standardizes its behavioral telemetry early will spend less to comply, and is in a better position to push its standard upward into the regulations rather than absorb whatever schema regulators invent.

On integration. TAO tuples can be emitted from the tool-call layer using existing trace and span data as input. The standard does not require model changes, retraining, or access to weights. Phase 1 of the pilot path adds a tuple writer alongside the existing telemetry; no other layer of the agent stack changes.

What to read next

Five more minutes: *TAO_ADOPTION_BRIEF.md*, the two-page pitch with a worked scenario and a five-phase pilot path. — **Thirty minutes:** *TAO_v0_11.md* abstract plus §1, §2, §6, §10. — **Two hours, with intent to build:** the full spec, then *REFERENCE_VALIDATOR_SPEC.md*, then the test vectors. — **Regulatory or audit work:** *TAO_COMPLIANCE_CROSSWALK.md* maps TAO features to EU AI Act, NIST AI RMF, ISO 42001, SOC 2, PCI DSS.

Status and ask

Status. Working draft. The structural pieces — tuple format, kernel, conformance levels, override discipline — are intended to be stable across the 0.x line. The verb vocabulary will move. A runnable reference validator is the next planned artifact; until it ships, the test vector suite is the authoritative conformance criterion.

Ask. I'm coming at this as an outside systems builder, not as someone working inside a frontier lab. I am looking for hard critique on where this breaks against real agent infrastructure: where the nine-effect kernel is too coarse for an action class you have shipped, where the verb vocabulary breaks against something concrete you have seen, where the observer-independence levels don't match how your platform actually isolates things, and whether the override discipline would catch the laundering you have already had to hand-investigate.